

The Clarke & Wright Savings Heuristic for the Capacitated Vehicle Routing Problem: Sequential and Parallel Versions

Heuristic Algorithms in Transport and Finance — Master's course assignment

Javier Salvatierra

Jesús Inurria

September 2026

Abstract

The Capacitated Vehicle Routing Problem (CVRP) asks for a set of minimum-cost routes that start and end at a depot, visit every customer exactly once and never exceed the capacity of a vehicle. We describe the savings heuristic of Clarke and Wright (CWS), implement its sequential and parallel versions in Python — the parallel one also in C++ — and test them on the 33 classical benchmark instances used by Juan et al. (2011). The parallel CWS reproduces the CWS cost reported by Juan et al. in 27 of the 33 instances (all 33 with tied savings ordered the other way round), 4.91 % above the best-known solutions on average; the sequential one is 11.6 % worse. Both take milliseconds in Python, and the C++ port returns the same solutions in 13 μ s–1.4 ms.

1 Introduction

The Vehicle Routing Problem (VRP) generalises the Travelling Salesman Problem (TSP). In the TSP a single traveller visits every city once along the shortest closed tour. In the VRP a *fleet* of vehicles leaves a common depot, every customer is visited by exactly one vehicle and every vehicle returns to the depot: one must decide how the customers are split into routes and in which order each route visits them, minimising the total distance.

In the *capacitated* version (CVRP) the vehicles carry goods: each vehicle has capacity Q and each customer i consumes a demand q_i from it, so the total demand served by a route cannot exceed Q . The CVRP contains the TSP as the special case of a single vehicle with unlimited capacity, and it is therefore NP-hard, which has motivated many heuristics that find near-optimal solutions quickly. The savings heuristic of Clarke and Wright [1] (CWS) is probably the most cited of them [3]: it is simple, has no parameters, and gives solutions typically 5–10 % above the best known in milliseconds [2].

We test the heuristic on the 33 classical CVRP instances selected by Juan et al. [3], which come from six classical families (A, B, E, F, M and P), with random, clustered and real-world customer layouts. An instance named A-n32-k5 has $n = 32$ nodes (the depot and 31 customers) and a best-known solution with $k = 5$ vehicles; sizes range from 22 to 135 nodes. The depot is central in some instances and in a corner in others. Each file gives the coordinates and the demand of every node, with the depot translated to the origin; the capacity Q and the reference costs are taken from Table 1 of [3]. Below we state the model, describe the CWS and our implementation, and compare the results with that table.

2 Problem formulation and savings

Let node 0 be the depot and $N = \{1, \dots, n-1\}$ the set of customers. Every customer i has a demand $0 < q_i \leq Q$, and $c(i, j) = c(j, i)$ is the Euclidean distance between nodes i and j , not rounded. The fleet consists of identical vehicles of capacity Q ; the CWS does not fix their number, so a solution may use more vehicles than the k of the instance name. A solution is a set of routes $R_1, \dots, R_m$, each an ordered sequence of customers travelled as $0 \rightarrow R_r \rightarrow 0$, and the CVRP reads

$$\min \sum_{r=1}^m c(R_r) \quad \text{s.t.} \quad R_1 \cup \dots \cup R_m = N, \quad R_r \cap R_s = \emptyset \ (r \neq s), \quad \sum_{i \in R_r} q_i \leq Q \quad (r = 1, \dots, m), \quad (1)$$

where $c(R_r)$ is the length of the closed tour of route r : every customer is visited exactly once and no route is overloaded.

Serving every customer with its own vehicle, $0 \rightarrow i \rightarrow 0$, costs $2c(0, i)$ per customer (there and back), $2 \sum_{i \in N} c(0, i)$ in total. If two customers i and j are instead served consecutively by the same vehicle, the return $i \rightarrow 0$ and the departure $0 \rightarrow j$ are replaced by the direct edge $i \rightarrow j$, which saves

$$s(i, j) = c(0, i) + c(0, j) - c(i, j). \quad (2)$$

By the triangle inequality, $c(i, j) \leq c(i, 0) + c(0, j)$, so $s(i, j) \geq 0$: going directly from one customer to the next is never longer than going back to the depot in between. Savings are largest for customers that are close to each other and far from the depot.

3 The Clarke and Wright savings heuristic

Clarke and Wright [1] build a solution by starting from the dummy solution above and merging routes along the edges with the largest savings, as long as the constraints of (1) hold.

3.1 Dummy solution

The initial solution assigns one vehicle to each customer. It is feasible because $q_i \leq Q$, every customer is visited once and receives its demand q_i , and it has the largest possible cost. The savings $s(i, j)$ of all $\binom{n-1}{2}$ pairs of customers are computed and sorted in decreasing order into the *savings list*.

3.2 Improving the dummy solution by merging routes

The heuristic walks down the savings list. For an edge (i, j) , the route that contains i and the route that contains j are merged into one route that goes through $i \rightarrow j$ if three conditions hold:

1. **Different routes.** If i and j were already on the same route, the new edge would close a cycle that does not visit the depot.
2. **Both nodes are exterior,** i.e. adjacent to the depot (first or last customer of their route). An interior customer already has two neighbours and cannot receive a third one.
3. **Capacity.** The joint load of the two routes does not exceed Q .

A merge replaces $(i, 0)$ and $(0, j)$ by (i, j) , so the cost drops by exactly $s(i, j) \geq 0$; taking the edges in decreasing order of saving makes the heuristic greedy. There are two classical ways of walking the list [1, 3]:

- **Parallel CWS.** The list is walked once, and *any* two routes are merged whenever the conditions hold, so all routes grow at the same time. This is the version used by Juan et al. because it usually gives better results [3].
- **Sequential CWS.** One route is grown at a time: it is extended with the best feasible edge that touches one of its two ends until none is left, then it is closed and the next route is started.

In both versions an edge that fails the conditions can be discarded for good: routes only grow, so a node that became interior stays interior, two nodes on the same route stay together and loads only increase.

4 Implementation

Juan et al. [3] build SR-GCWS-CS on top of the CWS: a multi-start method that picks the next edge at random (geometric distribution) from the top of the list, caches the best order of every route and splits the plane into sub-problems. We implement only the classic, deterministic CWS, lines 1–2 of their Figure 4 (savings list and CWS solution); our loop has the structure of their Figure 5 but always takes the first edge. The reference is thus the column *CWS Sol.* of their Table 1, not *Our Best Sol.*

The code (Appendix ??) is a Python 3.14 module, `vrp.py`, built on NumPy, plus a C++ port of the parallel version. Distances are computed once as an $n \times n$ matrix, and the savings list is sorted with a *stable* sort, so edges with equal saving keep their generation order (i, j) with $i < j$, as in the course code [2].

4.1 Common objects: the dummy solution

Both versions start from `construct_initial_sol`, which builds three objects kept up to date by the algorithm: `routes`, a dictionary *route index* $\rightarrow$ *list of customers* (the depot at both ends is implicit; initially route i is $[i]$); `route_of`, the route index of every node, so that the route of a customer is found in $O(1)$; and `load`, the demand carried by each route, so that the capacity check is $O(1)$. The conditions of Section 3 are checked by `check_merging_conditions`. A merge (`merge_routes_using_edge`) reverses the routes if needed so that the first one ends at i and the second one starts at j , concatenates them, relabels the customers of the second route in `route_of` and adds up the loads.

4.2 Sequential version

`sequential_cws` seeds a route with the first feasible edge of the list, then scans the list again from the top for the first feasible edge with one end on the current route, merges, and repeats. When no edge can extend the route, the route is closed and a new one is seeded; the heuristic stops when no feasible edge is left. Edges that touch the current route and fail the conditions are deleted; edges that do not touch it are kept for the routes still to come.

4.3 Parallel version in Python and C++

`parallel_cws` reuses the same building blocks and differs only in the way the list is walked: each edge is tried exactly once, in decreasing order of saving, and merges the routes of its two nodes, whichever they are, if the conditions hold. The C++ file `cws.cpp` ports it function by function (`std::stable_sort` for the list) and is compiled with Clang through Zig (`zig c++ -O2 -ffp-contract=off`). Disabling fused multiply-add keeps every floating-point operation identical to NumPy, so savings, ties and costs match bit for bit: the 33 solutions are the same.

5 Computational results

Table 1 compares both versions with Table 1 of [3], with $\text{gap} = 100(\text{cost} - \text{ref})/\text{ref}$. Times cover the savings list and the construction: one run in Python, the mean of repeated runs over at least 0.1 s in C++.

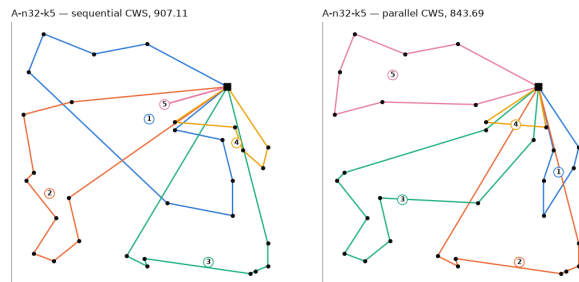


Figure 1: A-n32-k5 (square: depot). The last route of the sequential CWS (left) serves a single left-over customer; the parallel CWS (right) matches the JORS cost.

Table 1: Sequential and parallel CWS. *BKS* (real distances) and *JORS CWS* from Table 1 of [3]; routes in bold exceed k ; times of the parallel CWS.

Instance	Q	BKS	JORS CWS	Seq. cost	Seq. vs JORS (%)	Par. cost	Par. vs JORS (%)	Par. vs BKS (%)	Par. routes	Python (ms)	C++ (µs)
A-n32-k5	100	787.81	843.69	907.11	7.52	843.69	0.00	7.09	5	1.23	38
A-n38-k5	100	734.18	768.14	834.26	8.61	768.13	0.00	4.62	6	1.61	38
A-n45-k7	100	1147.28	1199.98	1251.38	4.28	1199.98	0.00	4.59	7	2.20	60
A-n55-k9	100	1074.46	1099.84	1290.89	17.37	1099.84	0.00	2.36	9	3.12	125
A-n60-k9	100	1355.80	1421.88	1572.97	10.63	1421.88	0.00	4.87	9	3.66	158
A-n61-k9	100	1039.08	1102.23	1188.96	7.87	1102.23	0.00	6.08	10	3.95	165
A-n65-k9	100	1181.69	1239.42	1479.40	19.36	1239.42	0.00	4.89	10	4.25	234
A-n80-k10	100	1766.50	1860.94	1956.41	5.13	1860.94	0.00	5.35	10	6.23	422
B-n50-k7	100	744.78	748.80	849.34	13.43	748.80	0.00	0.54	7	2.59	82
B-n52-k7	100	750.08	764.90	839.92	9.81	764.90	0.00	1.98	7	2.72	104
B-n57-k9	100	1603.63	1653.42	1769.42	7.02	1653.42	0.00	3.11	9	3.37	136
B-n78-k10	100	1229.27	1264.56	1401.39	10.82	1264.56	0.00	2.87	10	5.96	376
E-n22-k4	6000	375.28	388.77	448.90	15.47	388.77	0.00	3.60	4	0.70	13
E-n30-k3	4500	535.80	534.45	611.05	14.33	534.45	0.00	-0.25	4	1.03	33
E-n33-k4	8000	838.72	843.10	913.12	8.31	843.10	0.00	0.52	4	1.28	40
E-n51-k5	160	524.94	584.64	625.56	7.00	584.64	0.00	11.37	6	2.63	93
E-n76-k7	220	687.60	737.74	862.67	16.93	738.13	0.05	7.35	7	5.49	333
E-n76-k10	140	837.36	900.26	1005.25	11.66	907.39	0.79	8.36	10	5.64	337
E-n76-k14	100	1026.71	1073.43	1161.29	8.18	1054.60	-1.75	2.72	15	6.01	363
F-n45-k4	2010	724.57	739.02	1003.09	35.73	739.02	0.00	1.99	4	2.08	71
F-n72-k4	30000	248.81	256.19	346.47	35.24	256.19	0.00	2.96	5	4.60	291
F-n135-k7	2210	1170.65	1219.32	1423.00	16.70	1219.32	0.00	4.16	7	15.95	1382
M-n101-k10	200	819.81	833.51	939.99	12.78	833.51	0.00	1.67	10	9.38	687
M-n121-k7	200	1045.16	1068.14	1291.33	20.90	1068.14	0.00	2.20	7	12.70	1011
P-n22-k8	3000	601.42	590.62	625.91	5.97	590.62	0.00	-1.80	9	0.77	13
P-n40-k5	140	461.73	518.37	516.42	-0.38	518.37	0.00	12.27	5	1.81	44
P-n50-k10	100	699.56	734.32	740.57	0.85	739.84	0.75	5.76	11	2.74	86
P-n55-k15	70	991.48	978.07	1036.48	5.97	978.07	0.00	-1.35	17	3.56	113
P-n65-k10	130	796.67	851.67	938.51	10.20	851.67	0.00	6.90	10	4.33	196
P-n70-k10	135	830.02	896.86	957.87	6.80	896.86	0.00	8.05	11	4.86	261
P-n76-k4	350	598.22	689.13	724.08	5.07	688.34	-0.11	15.07	4	5.15	301
P-n76-k5	280	635.04	698.51	793.37	13.58	709.38	1.56	11.71	5	5.28	298
P-n101-k4	400	692.28	765.38	843.70	10.23	765.38	0.00	10.56	4	8.72	692
Mean					11.62		0.04	4.91		4.41	261

Parallel CWS vs. JORS. Our parallel CWS gives exactly the JORS cost in 27 of the 33 instances. The other six (E-n76-k7, E-n76-k10, E-n76-k14, P-n50-k10, P-n76-k4, P-n76-k5) differ by between -1.75% and $+1.56\%$, and the mean gap is 0.04% . The difference comes only from the order of edges with equal saving, which the CWS leaves undefined and which is frequent here because the coordinates are integers: if tied edges are taken in reverse generation order, the same code reproduces the JORS cost in all 33 instances. Neither rule is better in general — ours gives the lower cost on E-n76-k14 and P-n76-k4, the reversed one on the other four — which shows how sensitive a greedy construction is to arbitrary choices, the very idea that Juan et al. exploit with randomisation.

Quality. The mean gap of the parallel CWS to the best-known solutions is 4.91% (4.87% for the JORS CWS), within the $5\text{--}10\%$ usually quoted for the heuristic [2], with family means from 1.93% (M) to 7.46% (P). In three instances (E-n30-k3, P-n22-k8, P-n55-k15) the gap is negative because the CWS uses more vehicles than the best-known solution (one or two more), which is allowed here; overall, 11 of the 33 solutions use more routes than k .

Sequential vs. parallel. The sequential CWS is on average 11.62% worse than the JORS CWS and better only in P-n40-k5 (-0.38%). Its first routes pick the best savings freely, but the last ones are built from the customers left over, which are often scattered or even alone (Figure 1). The penalty depends on the family: 6.48% for P, $10\text{--}12\%$ for A, B and E, 16.84% for M and 29.23% for the clustered instances F.

Time. The parallel CWS needs $0.70\text{--}15.95$ ms in Python, dominated by sorting the $\binom{n-1}{2}$ savings; the sequential one up to 78 ms, because it rescans the list for every merge. The C++ port takes $13\mu\text{s}\text{--}1.38$ ms, 12 to 58 times less (median 24; one Python run against a mean of many C++ runs, so only indicative). Such a cheap construction is what makes multi-start methods like SR-GCWS-CS affordable.

6 Conclusions

The CWS turns the CVRP into one greedy pass over a sorted list of savings: no parameters, always feasible and, in its parallel version, about 5% above the best-known solutions in milliseconds. Our implementation reproduces the CWS column of Juan et al. once the tie rule is matched, and the C++ port gives the same solutions $12\text{--}58$ times faster. The sequential version is about 10% worse, above all on clustered instances. The sensitivity to tied savings points to the next step: randomising the choice of edges and repeating the construction, as Juan et al.

References

- [1] G. Clarke and J. W. Wright. Scheduling of vehicles from a central depot to a number of delivery points. *Oper. Res.*, 12(4):568–581, 1964.
- [2] A. A. Juan. *The VRP and the CWS heuristic*. Course slides, Heuristic Algorithms in Transport and Finance, 2026.
- [3] A. A. Juan, J. Faulín, J. Jorba, D. Riera, D. Masip and B. Barrios. On the use of Monte Carlo simulation, cache and splitting techniques to improve the Clarke and Wright savings heuristics. *J. Oper. Res. Soc.*, 62(6):1085–1097, 2011.